

引擎室里都有什么

代码、项目与算法的完整通俗详解

(详尽到能给开发者看,自然到零基础也读得下去)

Chladni 板逆向设计系统 —— 帝国理工学院物理系

Contents

1 这套东西到底在干什么 (一口气说完)	1
2 代码地图	2
2.1 入口与配置	2
2.2 应用层: 前端 + 后端桥	2
2.3 算法核心 (本文重点)	2
2.4 支撑机件	2
3 算法 1——从手绘到一句诚实的裁决	3
4 算法 2——对称性守门人 (D_4 与那个可达性数字)	3
5 算法 3——快速物理"草稿" (可微代理)	4
6 算法 4——材料的小花招, 以及我们为什么把它冻住	4
7 算法 5——像真沙子那样打分 (可识别度评分)	5
8 算法 6——搜索引擎们	5
9 算法 7——两阶段生产管线 (面向客户的主路径)	6
10 算法 8——和真实仿真器对话 (COMSOL 桥)	6
11 算法 9——从设计到能拿在手里的东西 (STL 导出)	6
12 把它们拼起来: 一次完整的运行	7
13 怎么自己跑	7
14 小词典	7

1、这套东西到底在干什么 (一口气说完)

你画一个目标图案。软件去设计一块方板——方法是给一个 15×15 网格上的每个点选一个厚度——再选一个振动频率,使得真实的板从中心被摇、撒上沙子后,沙子聚成一幅像你目标的图案。最关键的是,它还会在你白费力气之前,先告诉你这幅图在物理上到底可不可能。

数据是单向流动的,经过一条界限清晰的步骤链:

你的手绘 → 干净目标 + "这可能吗?" 裁决 → 快速代理搜索 (厚度 + 频率) → 精确 COMSOL 仿真 (挑出最佳振动) → 信赖域微调 → 用"撒沙物理"打分 → 可打印 STL + 报告

`src/` 里几乎每个文件夹都是这条链上的一环。本文就一环一环地走:先是代码的地图 (第 2 节),再是每个算法各自一节,然后是完整的端到端走查 (第 12 节) 和怎么跑 (第 13 节)。

2、代码地图

这个项目是一个 Python 包 (`src/`) 加一个网页前端 (`frontend/`)、一组 COMSOL/MATLAB 模板 (`comsol_templates/`)，以及数据与报告目录。下面说明每一部分的用途。

2.1 入口与配置

- `src/main.py`——命令行总入口。子命令包括 `target-ui` (启动网页应用)、`prepare-target`、`generate-candidates`、`run-workflow`、`diagnose-comsol`、`self-test`。
- `config.yaml` + `src/config.py`——一个集中的设置文件 (板尺寸、网格大小、材料参数、评分权重、COMSOL/MATLAB 路径)，只读一次，然后作为一个 `config` 字典发给每个模块。想改板子或材料，就改这里，而不是改算法。

2.2 应用层：前端 + 后端桥

- `frontend/target_designer.html`——一个单页应用，含 `Target` / `Tune` / `Results` / `Run` / `Gallery` / `Export` 等页签。它只是按钮、画布和 JavaScript，从不直接碰 COMSOL。
- `src/frontend/target_ui_server.py`——页面背后的 Python 网页服务 (约 2,500 行)。它负责跑管线、管理 COMSOL 连接、推送进度、可用 `Stop` 按钮中途取消、并提供结果。可以把它理解为 HTML 与算法之间的桥。

2.3 算法核心 (本文重点)

- `src/target/`——把手绘变成干净的黑白目标，并判断它是否可实现 (第 3 节)。
- `src/symmetry/`——“对称性”守门人：产生那个至关重要的可达性数字的 D_4 分解 (第 4 节)。
- `src/physics/`——快速、可微的板“草稿”，以及它的正交各向异性 (带方向材料) 版本 (第 5–6 节)。
- `src/scoring/`——一幅图案如何被打分，从早期的像素重叠到撒粉物理的“可识别度”评分 (第 7 节)。
- `src/optimisation/`——真正的搜索引擎，从随机搜索一直到 W10 梯度优化器与两阶段生产管线 (第 8–9 节)。
- `src/comsol/`——一切通过 MATLAB LiveLink 与真实 COMSOL 仿真器对话的代码 (第 10 节)。
- `src/export/`——把做好的厚度场变成可打印的 STL (第 11 节)。

2.4 支撑机件

- `src/forced_response/`、`src/frequency_domain/`、`src/nodal/`——在某个频率上驱动板、读取响应，并从仿真场里提取节线。
- `src/subspace/`——数据驱动的设计空间 (历史候选的 PCA “流形”、模态基)，供更聪明的搜索使用。
- `src/uniform_pattern/`——一份均匀板振动模态的目录，可浏览、可轻微微扰，并按“中心激振到底能不能激发它”来过滤。
- `src/topology_grammar/`、`src/tags/`、`src/candidate/`——生成厚度候选并施加约束 (把厚度保持在 0.6–2.0 mm，并满足相邻差限制)。

- `src/visualisation/`——生成你在 UI 里看到的预览图。
- `comsol_templates/`——参数化的 COMSOL 模型 (`.mph`) 和 LiveLink 为每个候选运行的 MATLAB 脚本。

3、 算法 1——从手绘到一句诚实的裁决

文件夹：`src/target/`

一句人话：动手造任何东西之前，好工程师都会先问“这到底可不可能？”。这第一步会把你的草图清理干净，然后给它一个直白的体检结论：容易、需要改拓扑，或者别想了。

实际发生了什么。`preprocess_target.py` 把你画的或上传的任何图，转成一张干净的二值图（前景 = 你想要节线的地方）。`analyse_target.py` 量一些基本健康指标：画布被填了多少、有多少独立块、笔画有多细。

然后 `target_realizability.py` 综合若干有物理依据的“红旗”，给出一句**裁决**：

- 一个 **Courant 指数**——粗略地说就是“要画出这么多独立区域，你得往振动模态表里翻到多高？”（基于经典的节点域计数定理）。太高 = 不现实。
- 一个 **笔画间隙检查**——节线之间不可能挨得比物理允许的更近；又细又密的笔画会被标红。
- 一个 **D_4 对称失配**——这幅图离板子的四重对称有多远（它是第 4 节那条深层结论的廉价预览）。
- 连通块数、内部空洞数、骨架端点数。

那些阈值（比如“Courant 指数 > 60 判不可达”，“ D_4 失配 > 0.55 判需要改拓扑”）都集中写在文件顶部的一个字典里，所以裁决透明、可调。

为什么重要。整个项目最大的教训，就是这句裁决应当被前置使用。“IC” 两个字母从一开始就注定无解，而这个模块在毫秒级就能说出来——远早于任何昂贵的仿真。

4、 算法 2——对称性守门人（ D_4 与那个可达性数字）

文件：`src/symmetry/d4_decomposition.py`（约 100 行，是整个项目的数学心脏）

一句人话：一块从正中心摇的方板，永远只能做出和方形同样对称的图案。这个模块衡量你的目标里有多少成分具备那种对称——而这个比例，就是你做得再好也越不过去的硬天花板。

实际发生了什么。方形的对称群 D_4 有八个动作：四个旋转（ $0^\circ, 90^\circ, 180^\circ, 270^\circ$ ）和四个镜面翻转（水平、垂直、两条对角线）。代码把每个动作都实现成简单的数组运算（`np.rot90`、`np.flipud`、转置）。

任何一幅图 P 都能被拆成五种“对称口味”（即不可约表示 A_1, A_2, B_1, B_2, E ），用的是一条标准的投影公式。对每种口味 ρ ，代码计算

$$P_\rho = \frac{\dim \rho}{8} \sum_{g \in D_4} \chi_\rho(g) (g \cdot P),$$

也就是：把八个动作全部作用一遍，每个用**特标表**里的一个 ± 1 （或 $0, 2$ ）加权，然后相加。这五块彼此完全正交，加回去就是原图（文件里甚至自带一个 `check_orthogonality` 自检）。

最关键的是一个函数 `coupling_accessibility`：中心点激振只能激发完全对称的那种口味 A_1 ，于是

$$\text{可达性}(P) = \frac{\|P_{A_1}\|^2}{\|P\|^2}.$$

对 X 形这是 100%；对单对角线约 52%；对”IC” 约 42%。图里剩下的能量都待在中心激振根本碰不到的口味里。

为什么重要。它把”我们的算法还不够好”变成了精确、可证明的”IC 有 58% 在物理上被禁止”。它是第 3 节那句裁决背后的数学骨架，也是整个项目诚实的底气。

5、 算法 3——“快速物理”草稿”（可微代理）

文件：`src/physics/differentiable_plate.py`

一句人话：要在几百万种板子设计里搜索，我们需要一个快办法来猜每块板怎么振动。这就是一个精简版的物理模型，零点几秒就跑完——而且很关键，它被造得能让计算机自动算出”我该往哪个方向微调厚度，才能让结果更好？”

实际发生了什么。板子被放到一个网格上（取奇数尺寸如 25×25 ，好有一个真正的中心格）。模型造出两样原料：

- 一个**刚度矩阵** K 和一个**质量** M 。刚度来自薄板弯曲定律：每格的弯曲刚度是 $D = \frac{E h^3}{12(1-\nu^2)}$ （越厚硬得多，因为有 h^3 ），而 $K = L^T \text{diag}(D) L$ ，其中 L 是一个简单的有限差分拉普拉斯算子。质量就是”每格坐落了多少材料”。
- **强迫响应。**以角频率 ω 摇中心、求解线性方程组 $(K + i\omega C - \omega^2 M)u = f$ ，就得到每点的复数振动幅 u 。 $|u|$ 小的地方沙子会聚集——那就是预测的 Chladni 图案。

中心的夹持，靠的是直接把那些网格点从求解里删掉。

整个模型用 PyTorch 写成，这意味着它是**可微的**：PyTorch 能自动算出任意分数对全部 225 个厚度值的梯度（**自动微分**）。正是这个梯度，让有方向的聪明搜索成为可能，而不必盲猜。

为什么重要——以及那个陷阱。这个模型是个快指南针，不是裁判。它系统性地过于乐观（把板子当作薄如纸片、忽略三维效应），所以它能承诺 8–18 倍的提升，而真实仿真只兑现一部分。整个项目的设计准则正源于此：用代理来选方向，绝不用它来宣布胜利。

6、 算法 4——材料的小花招，以及我们为什么把它冻住

文件：`src/physics/orthotropic_plate.py`；由 W10 使用

一句人话：3D 打印的塑料沿着打印纹路会稍微更强一点。我们做了一版让每格”纹路方向” θ 可以旋转的板模型，本指望它能帮我们绕开对称。结果没用，于是把它冻住了。

实际发生了什么。`orthotropic_plate.py` 组装同一类刚度矩阵，但用的是带方向的（**正交各向异性**）弯曲定律（出自 Reddy 的层合板理论）：每格都能把自己的刚度张量旋转一个角度 θ 。原则上，让 θ 逐格变化会打破完美对称，从而可能让中心激振够到第 4 节那些”被禁止”的口味。

为什么被冻住。在优化器实际使用的频段里，质量项 $\omega^2 M$ 以约五个数量级压倒刚度项 K ——而 θ 只通过 K 进入。于是它对结果的影响被埋在数值噪声之下（实测敏感度只有百万分之几）。更糟的是， θ 添了一个晃动的额外维度，优化器在上面白费力气。一个 10 次的实验（5 目标 $\times$ 2 材料）证实：普通各向同性 PLA 在多数目标上胜过”优化过”的碳纤维 PETG。所以在 `recognisability_placement_w10.py` 里，取向场被固定为零（材料仍是正交各向异性，只是每格都朝同一方向），优化器只动厚度 H 和频率 ω 。

7、 算法 5——像真沙子那样打分（可识别度评分）

文件：`src/scoring/recognisability_score.py`

一句人话：我们怎么给“它像不像目标”打分？早期我们比像素，结果那是把错的尺子。修正办法是：照沙子真实的行为去打分。

实际发生了什么。沙子堆在振幅接近零的地方。函数 `chladni_powder_density` 用一个高斯把振幅场变成预测的沙图， $\text{粉} = \exp(-(|u|/\sigma)^2)$ ，其中 $|u|$ 先除以它自己的最大值（这是个刻意的选择——早先一版是加一个小常数，结果悄悄毁掉了所有 COMSOL 结果，因为 COMSOL 的振幅在 10^{-12} 量级）。从这张沙图算出四个数：

- **富集度**——沙子在目标上的密度比平均高多少（头号指标；“3.76×”就是密 3.76 倍）。
- **召回**——目标线里有多大比例真的被低振幅谷覆盖。
- **对比度**——目标区沙密度对背景之比。
- **方向**——沙图主方向是否和目标一致（两者主轴之间的余弦）。

它们用固定权重合成一个**综合分**（0.40 富集 + 0.30 召回 + 0.20 对比 + 0.10 方向）。

为什么重要。从像素重叠（`src/scoring/metrics.py`，IoU/Dice）换到这套撒粉模型，把整个候选历史重新排了序——有些最好的设计，在错误的指标下被埋没了好几个月。

8、 算法 6——搜索引擎们

文件夹：`src/optimisation/`

一句人话：给定一个目标和一种打分办法，我们到底怎么找出一张好的厚度图？项目试过大约二十种方法；下面是活下来的那些。

这个文件夹是三个时代（盲走 → 结构化 → 可识别度）外加现代生产代码的博物馆：

- `random_search.py`——时代 I 基线：生成大量随机厚度图，留最好的。稳健，但只会找到通用团块。
- `gradient_optimizer.py`——关键升级：因为代理可微，我们能顺着梯度“下坡”。它还处理那个把厚度保持在 0.6–2.0 mm 的**重参数化技巧**（用 sigmoid 把无界数字映射进合法区间），以及一个邻格平滑惩罚，好让板子保持可打印。
- `homotopy.py`——从板子本来就想做的图案出发，把目标慢慢“morph”向你的目标；它卡在哪里，就是对可达性的一个诚实度量。
- `spectral_placement.py`——把板子若干个固有频率推到驱动频率附近聚集。
- `multifreq_placement.py`——同时在多个频率上驱动（后来发现这有点是幻觉式的花招）。
- `recognisability_placement_w10.py`——**现代优化器**。它用 Adam 梯度法（约 250 步）在正交各向异性代理上联合调厚度 H 和驱动频率 ω ，并用撒粉物理损失打分。这就是生产管线里的“指南针”阶段。
- `trust_region_w9.py`——第一次尝试代理 $\leftrightarrow$ COMSOL 的校正回路；结构对但参数欠调。它的思路被并入了下面管线的第二阶段。

9、 算法 7——两阶段生产管线（面向客户的主路径）

文件：`src/optimisation/production_pipeline.py`（约 1,100 行）

一句人话：这就是那个大大的”真的跑一遍”红按钮。它把快速搜索和精确仿真按正确顺序串起来，让快模型只负责出主意，让可信的仿真来拍板。

实际发生了什么。文件自己记录了它的八个阶段：

- S1. **准备目标**（从 PNG/NPY）。
- S2. **W10 代理搜索**——快速拿到一张好厚度 H 和候选频率（第 8 节）。
- S3. **COMSOL 特征分析**——向真实仿真器要板子约 30 个真实固有模式。
- S4. **阶段一——模态选择**——按”像不像目标”给这些模态排序，并另外试几个聪明的”偏共振拍点”频率。
- S5. **COMSOL 强迫响应**——在选中的频率上求真实振动（不是草稿）。
- S6. **复合搜索**——穷举至多三个模态的组合（RMS / MAX / SUM），留下打分最高的那个组合。
- S7. **阶段二——信赖域精修**（可选）——以真实仿真为真值，对 H 做小而安全的微调：代理提一步，COMSOL 复核，然后接受该步或收缩”信赖半径”。相邻迭代的模态用 MAC（模态置信判据）匹配。
- S8. **打包交付物**——`H.csv`、一个摘要 JSON、复合沙图、密度图，以及可视化。

为什么是这个形状。它干净地把可能出错的两件事分开：选哪个频率（靠直接在 COMSOL 真实共振点驱动来固定）和选哪个形状（靠信赖域的小微调来固定）。一个同时校正两者的朴素回路往往发散——这正是早先 `trust_region_w9.py` 教给我们的。一个协作式 Stop 钩子让 UI 能及时杀掉跑得很久的 COMSOL 子进程。

10、 算法 8——和真实仿真器对话（COMSOL 桥）

文件夹：`src/comsol/` + `comsol_templates/`

一句人话：可信的物理住在 COMSOL 里——一个通过 MATLAB 驱动的专业仿真器。这个文件夹就是那支外交使团，负责让 Python 和 COMSOL 在任何机器上都能可靠地对上话。

实际发生了什么。`discovery.py` 找到 COMSOL 和 MATLAB 装在哪（或是否已在运行）；`diagnostics.py` 检查本机是否就绪；`server.py` 启动或连接一个 COMSOL mphserver；`run_livelink.py` 把一个候选的参数交给 MATLAB 脚本 `run_chladni_candidate.m`，后者加载参数化模型 `Chladni_15x15_paramet` 运行它、导出频率与模态；`import_results.py` 再把这些导出读回 NumPy。

为什么重要。这是整个项目里最依赖部署环境的部分（依赖授权、路径，以及那个确切的 .mph 模型）。它被刻意隔离，好让上面的算法永远不必知道仿真到底是怎么跑起来的那些脏细节。

11、 算法 9——从设计到能拿在手里的东西（STL 导出）

文件：`src/export/stl_export.py`

一句人话：最终的厚度图不过是 225 个数字。这一步把它们变成一个能直接送进打印机的三维模型。

实际发生了什么。15 × 15 的每一格变成一个小长方体，10 mm × 10 mm 见方、高度等于该格的厚度，全部坐在 $z = 0$ 的平底上（好贴住打印床）。这 225 个长方体被写成一个二进制 STL“三角面片汤”（2,700 个三角形，约 132 KB），台阶可选高斯柔化。该模块刻意不用任何重型三维库（只用 NumPy），所以即使磁盘快满了也能跑。

12、把它们拼起来：一次完整的运行

假设你画了一个 X 并按下 Run：

1. 前端 (frontend/target_designer.html) 把你的手绘发给后端 (src/frontend/target_ui_server.py)
2. src/target/ 把它清理干净，src/symmetry/ 报告“可达性 $\approx 100\%$ ”——绿灯。
3. 生产管线 (src/optimisation/production_pipeline.py) 跑 W10 代理搜索 (src/physics/ + src/scoring/)，在几秒内拿到厚度图和候选频率。
4. 它向 COMSOL (src/comsol/) 要真实模态，挑出最像 X 的那些，并真实地驱动它们。
5. 它尝试各种组合，可选地用信赖域回路微调厚度，并用撒粉物理指标给一切打分。
6. 它写出一份报告，并经 src/export/stl_export.py 写出一个可打印的 .stl。Results 和 Export 页签把沙图给你看，并让你下载这块板。

若目标是“IC”，第 2 步则会改为警告你：这幅图大半在物理上够不着——替你省下后面整段运行。

13、怎么自己跑

一切都走 src/main.py：

```
python -m src.main self-test                # 检查安装
python -m src.main diagnose-comsol          # 检查 COMSOL/MATLAB 是否可达
python -m src.main target-ui                # 启动网页应用（最常用的入口）
```

网页应用是友好的那条路：Target 页画/传图，Tune 页设材料，Run 页启动管线（带实时进度条和 Stop 按钮），Results 页检视，Gallery 页浏览按“中心激振能否激发”过滤后的均匀板模态，Export 页下载可打印的板。进阶用户也可以直接调用 prepare-target、generate-candidates 和 run-workflow。

14、小词典

节线

板上几乎不动的线；沙子聚在这里。看得见的 Chladni 图案就是这些节线的集合。

代理

那个快速、简化的物理模型 (src/physics/)。搜索的指南针，不是最终裁判。

COMSOL

那个又慢又准的专业仿真器；真值裁判。

D_4 / 不可约表示

方形的对称群及其五种”对称口味”。中心激振只够得到完全对称的那一种 (A_1)。

可达性

你目标里落在 A_1 上的比例；做得再好也越不过去的硬天花板。

富集度

沙子在你目标线上的密度比平均高多少倍；项目的头号成功指标。

信赖域

一种安全步法：只做你当前”信得过”那么大的改动，若这次改动令人失望就收缩信赖。

Adam

W10 搜索所用的一种流行的基于梯度的优化器。

STL

打印机看得懂的三维文件格式。

本文是完整技术论文与科普单页的”通俗详解”伙伴。三者从三个深度覆盖同一个项目：深水区（论文）、浅水区（科普），以及引擎室（本文）。